MP-SPDZ

Ivan Visconti

General-Purpose Approach to Secure Multi-Party Computation

- It protects input-output privacy and ensures that the computed output is correct based on the parties' inputs and a function F known upfront.
- Two approaches:
  - White box: build an ad-hoc protocol for function F
    - PRO: performance
    - CONS: cryptographers must be involved
  - Black box: use a compiler that, given the description of F, produces programs executing general-purpose protocols for secure circuit evaluation
    - PRO: no need of cryptographers
    - CONS: the general-purpose approach could be insufficient

MP-SPDZ

- Among existing libraries, the one that includes the most solutions for the general-purpose approach is MP-SPDZ
- PROs
  - it is possible to select various algorithms and modes to generate and execute programs
  - any computable function F can be used
  - F can be described using a Python-like language
  - it can be used in a hybrid (gray-box) way, letting it handle the most difficult cryptographic aspects
- CONs
  - it is a library with limited support and installation may require resolving version conflicts with required libraries
  - documentation is concise and available examples are extremely limited

Compilation and Execution

- ./compile.py 2psi.mpc
- Creates a binary file describing in bytecode the circuit corresponding to the function specified in the .mpc source file
- It is then possible to run an executable that executes a 2-party computation protocol using this circuit as input, executing one party's role
- ./mascot-party.x -N 2 -p 0 2psi // from 1st shell
- ./mascot-party.x -N 2 -p 1 2psi // from 2nd shell
- Input can be interactive, but for faster execution it is preferrable to store it in predefined files
  - Player-Data/Input-Pj-0 (can be used by party j)

Example: 2psi.mpc

```
from Compiler.types import Array, sint
from Compiler.library import print_ln

n = 10
party0_set = Array(n, sint)
party1_set = Array(n, sint)
intersection = Array(n, sint)

for i in range(n):
    party0_set[i] = sint.get_input_from(0)
    party1_set[i] = sint.get_input_from(1)

for i in range(n):
    intersection[i] = 0
    for j in range(n):
                result = (party0_set[i] == party1_set[j])
                result = result * party0_set[i]
                intersection[i] = (result > 0).if_else(party0_set[i], intersection[i])

print_ln("Secure Set Intersection:")

for i in range(n):
    out = intersection[i].reveal()
    print_ln_if(out,"Element in intersection: %s", out)
```

Exercise

- Write the circuit for a 2PC aiming at checking some biometric correspondence (e.g., a vector of integers) with an error tolerance
- Write the circuit for a 2PC of PSI that can be used by python programs avoiding the quadratic overhead of the example in the previous slide (suggestion: if H(x) = H(y) then x=y when H is collision resistant...)